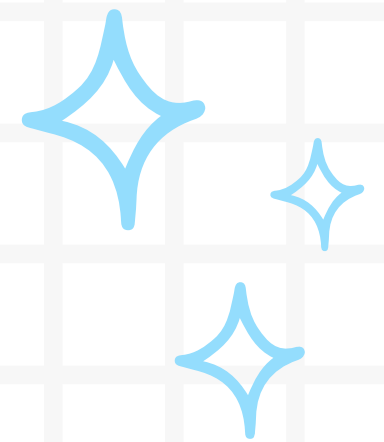
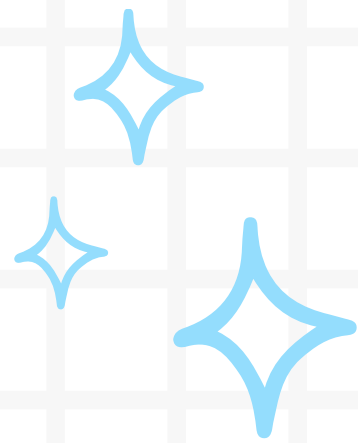


Los Makbets

RETO SEMANA 6

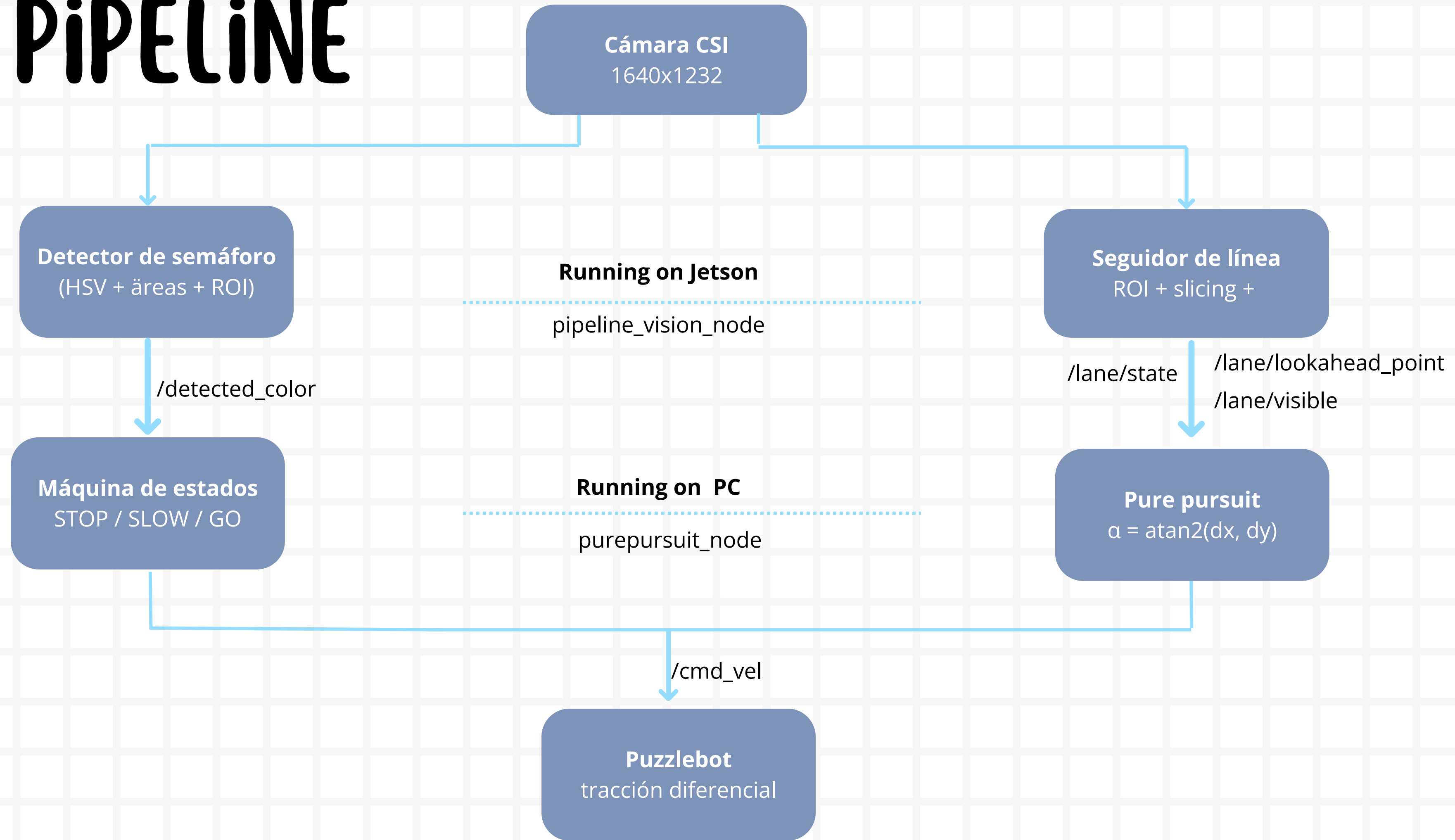




CONTENIDO

- | | | | |
|------------|--------------------|------------|-------------------|
| 01. | Pipeline | 04. | Cálculo del error |
| 02. | Detección de línea | | Estrategias de |
| 03. | Nodo controlador | 05. | recuperación |

PIPELINE



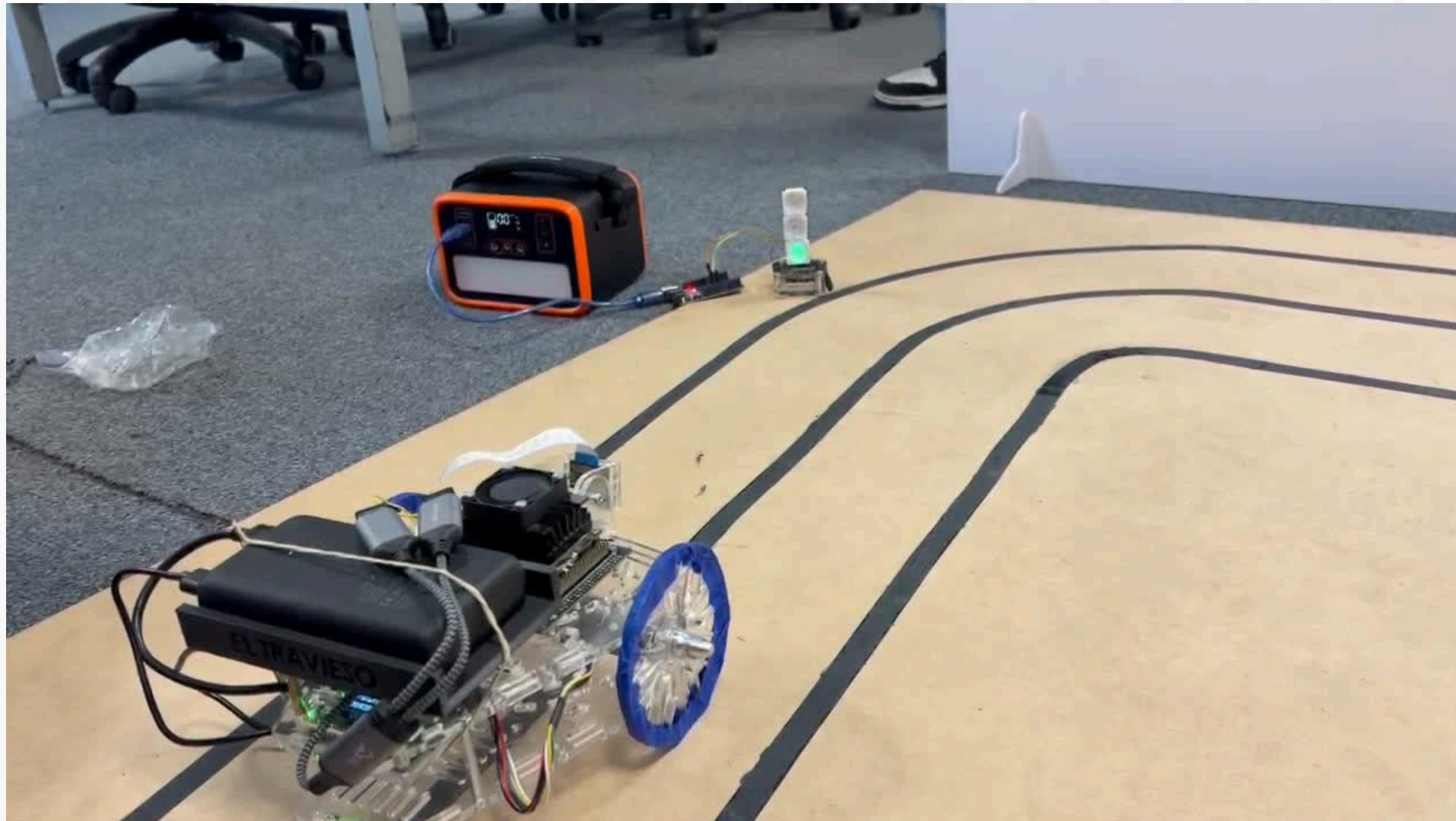
DETECCIÓN DE SEMÁFORO

HSV + máscaras por color con validación geométrica y umbralización por distancia.

- **ROI poligonal:** se recorta solo la región del frame donde se espera el semáforo),
- **Suavizado + cambio de espacio de color:** filtro Gaussiano 7×7 y conversión BGR → HSV Segmentación por color se generan tres máscaras binarias.
- **Verde:** 1 rango HSV
- **Amarillo:** 1 rango HSV
- **Rojo:** 2 rangos (el matiz cerca de 0° / 180° , hace falta cubrir ambos extremos)
- **Limpieza morfológica + validación geométrica del amarillo** (es la clase con más falsos positivos)
- **Filtros de contorno:** área 100 - 400 px, circularidad ≥ 0.6 ($4\pi A/P^2$)
- **Umbralización por área:** solo se acepta un color si su área supera un umbral calibrado a la distancia objetivo.

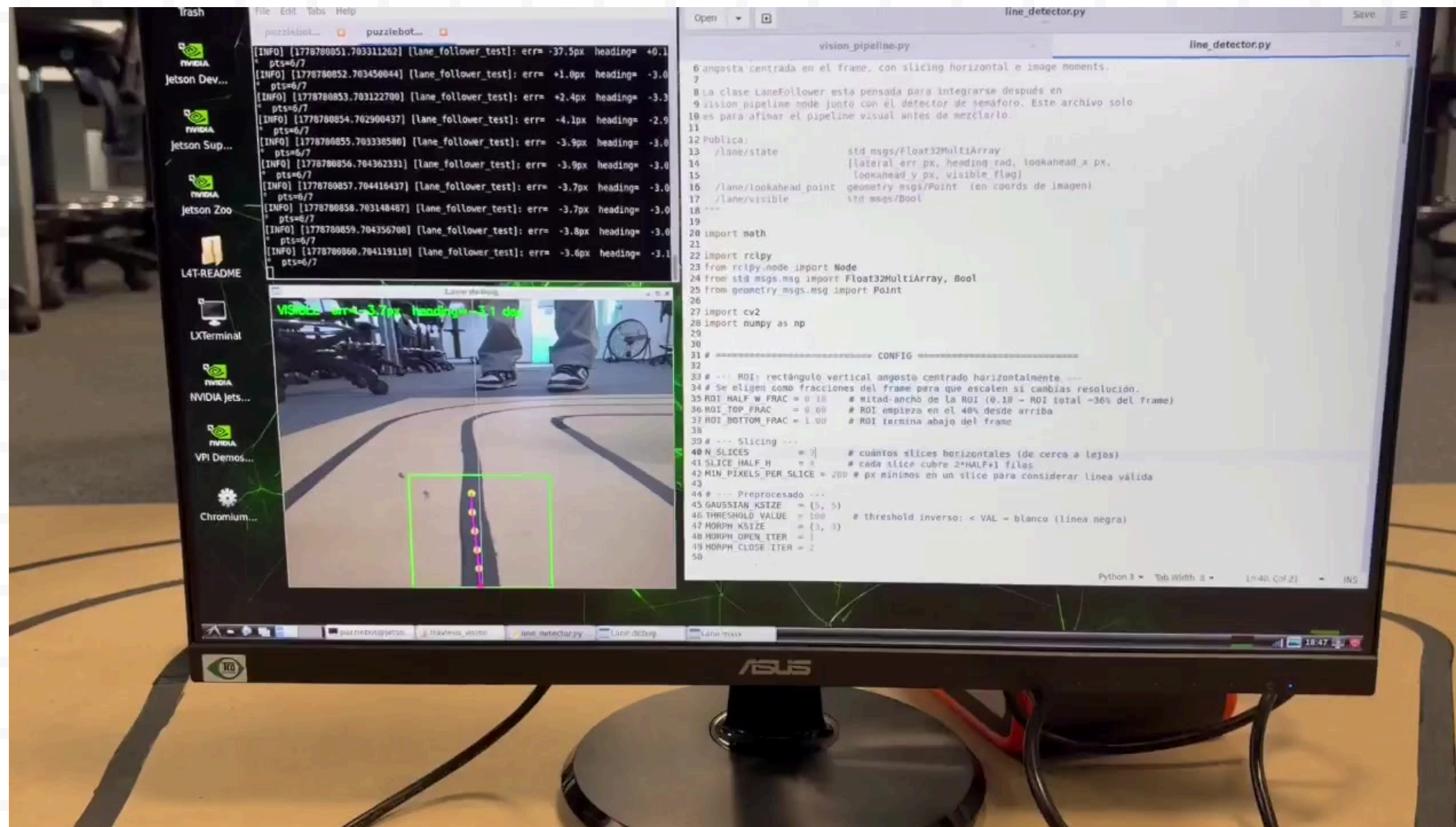
DETECCIÓN DE LÍNEAS Y SEMÁFORO

El nodo de detección de línea y detección del semáforo corre directamente en la Jetson.



DETECCIÓN DE LÍNEAS

El nodo de detección de línea corre directamente en la Jetson.

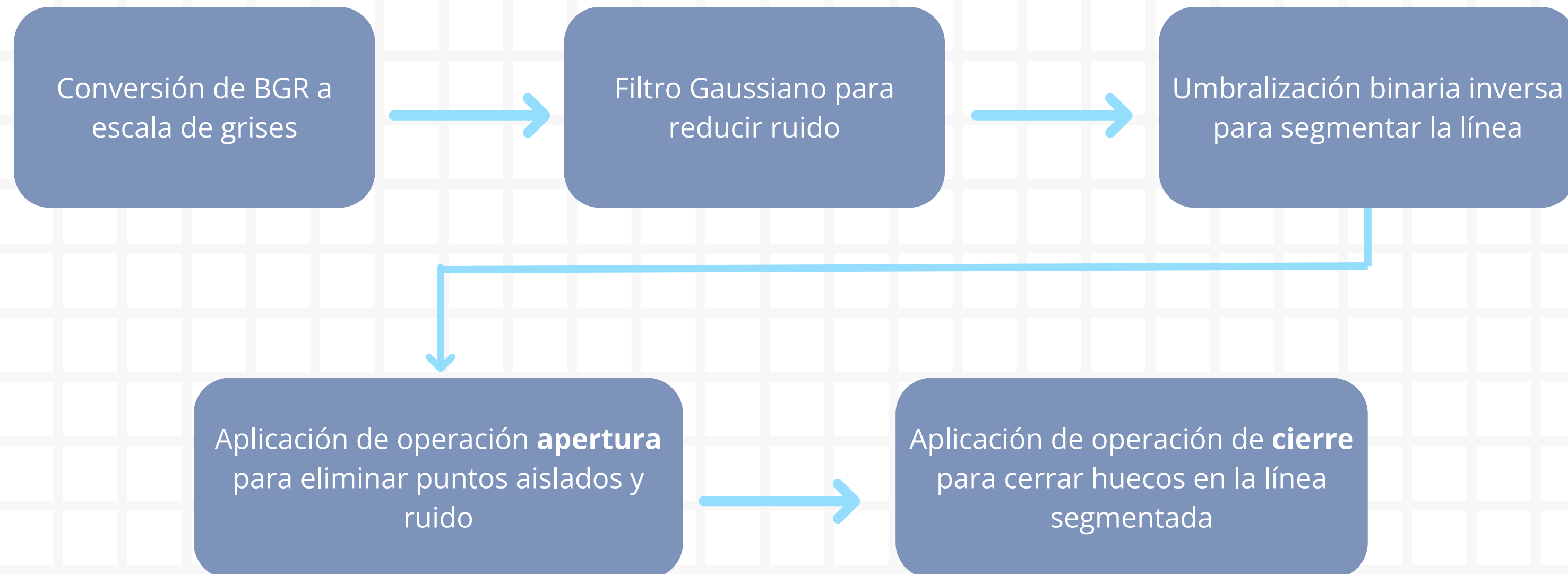


PRE-PROCESAMIENTO

Para mejorar la detección de la línea se aplica un pre-procesamiento sobre la ROI.

Parámetros actuales:

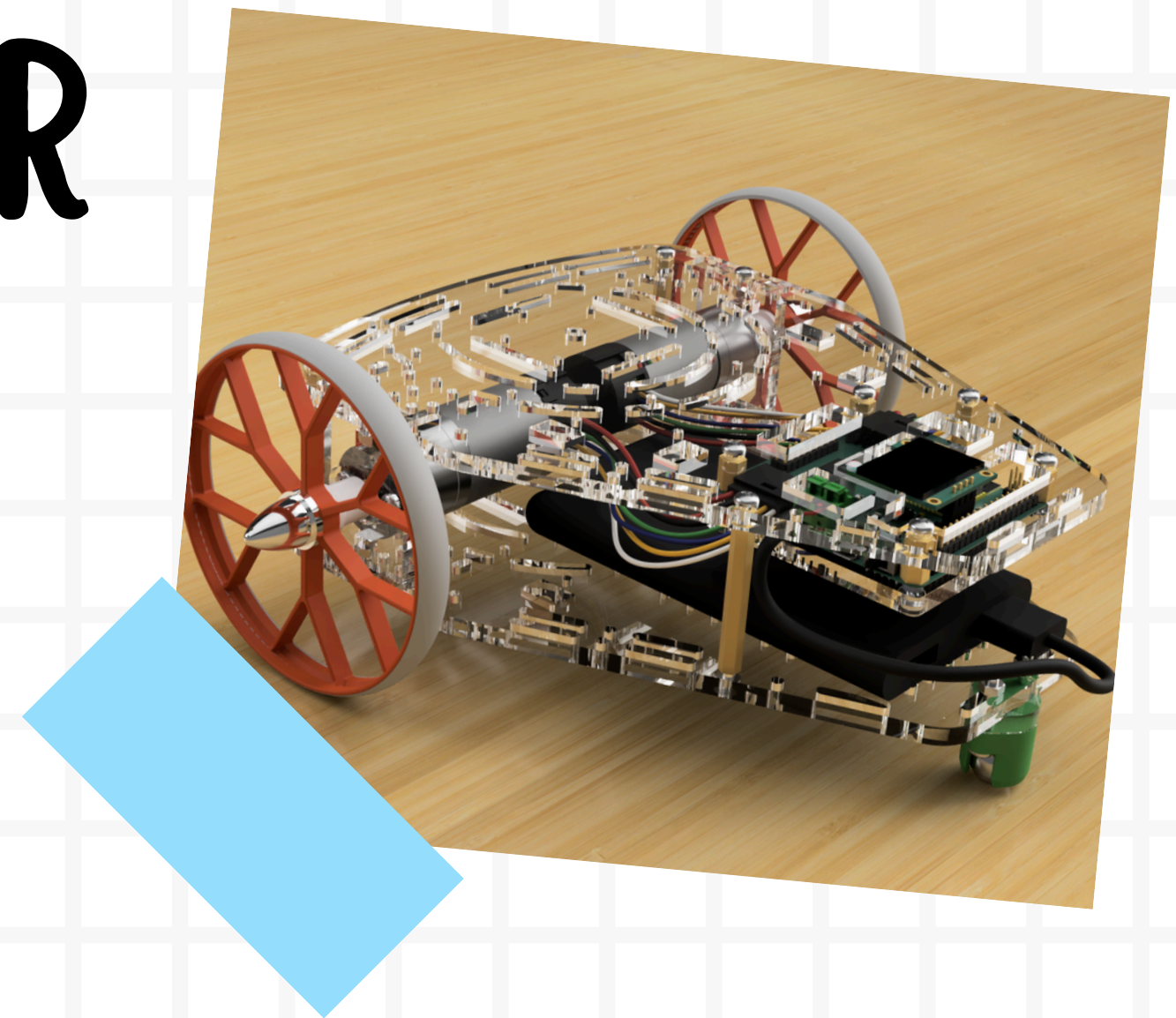
- threshold: 100
- kernel morfológico: 3x3
- open: 1
- close: 2



*Apertura: erosión y después dilatación.
Cierre: Dilatación y después erosión.

TIPO DE CONTROLADOR

Se utilizó un controlador (geométrico) de seguimiento de trayectorias *pure pursuit*, el cual recibe una referencia que el robot debe seguir y con ayuda de odometría geométrica corrige la posición actual con respecto al *look ahead* (donde se quiere llegar).



NODO CONTROLADOR

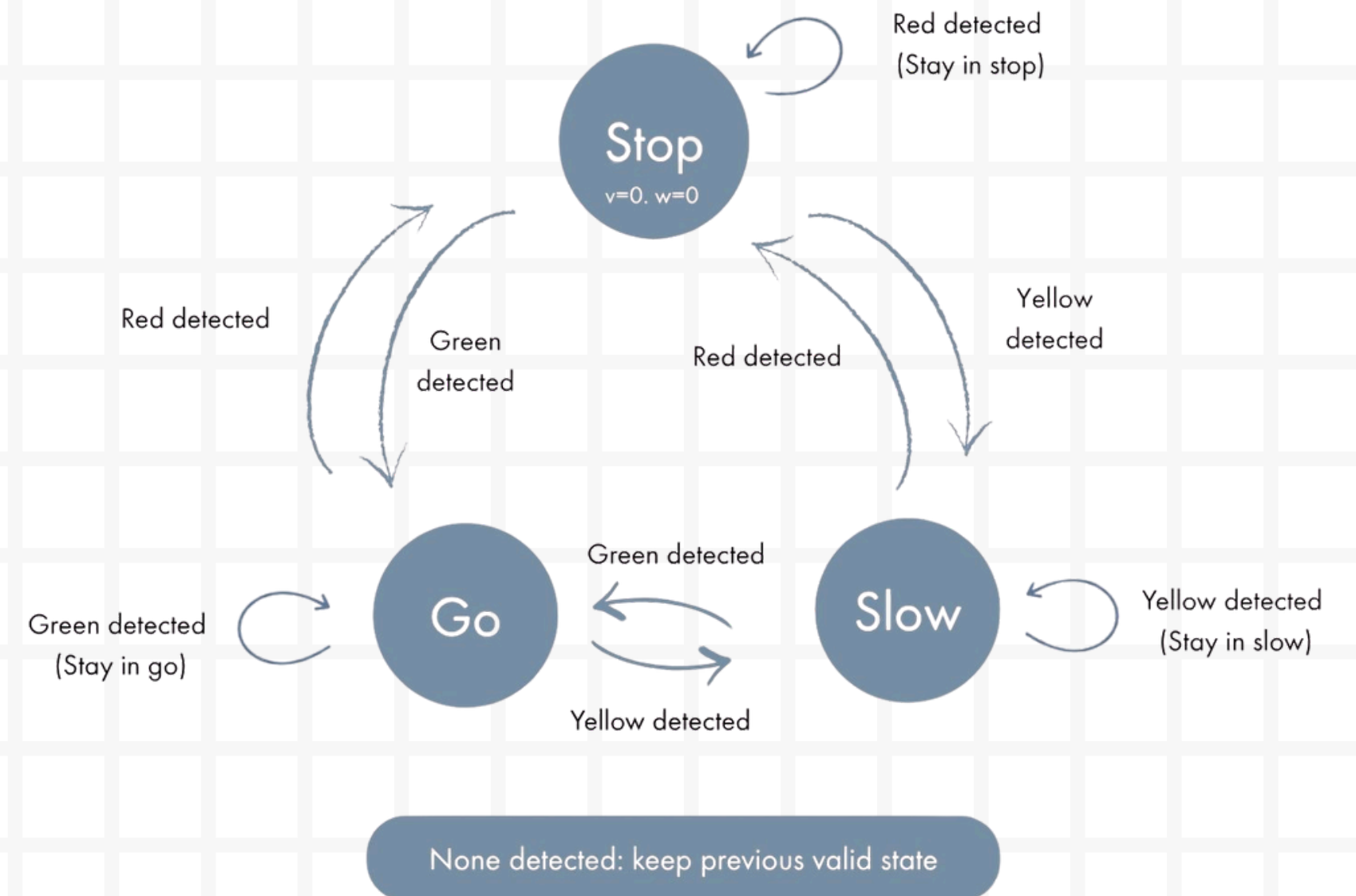
El nodo controlador se ejecuta desde la computadora local.

El controlador utiliza la información de visión para calcular la velocidad lineal y angular del robot. Además, incluye una máquina de estados para responder al semáforo.

se suscribe a:

- `/lane/lookahead_point`
- `/lane/visible`
- `/detected_color`

Publica los comandos de movimiento en : `/cmd_vel`



CÁLCULO DEL ERROR

El error se calcula con base en la posición del lookahead point respecto al centro de la imagen.

1

Se calcula el desplazamiento horizontal
(diferencia se expresa en px)

$$DX = LA_X - FRAME_WIDTH / 2$$

- la_x: coordenada x del lookahead.
- frame_width / 2 representa el centro de la imagen y es la odometría geométrica del robot

2

Se calcula la distancia vertical aproximada

$$DY = FRAME_HEIGHT - LA_Y$$

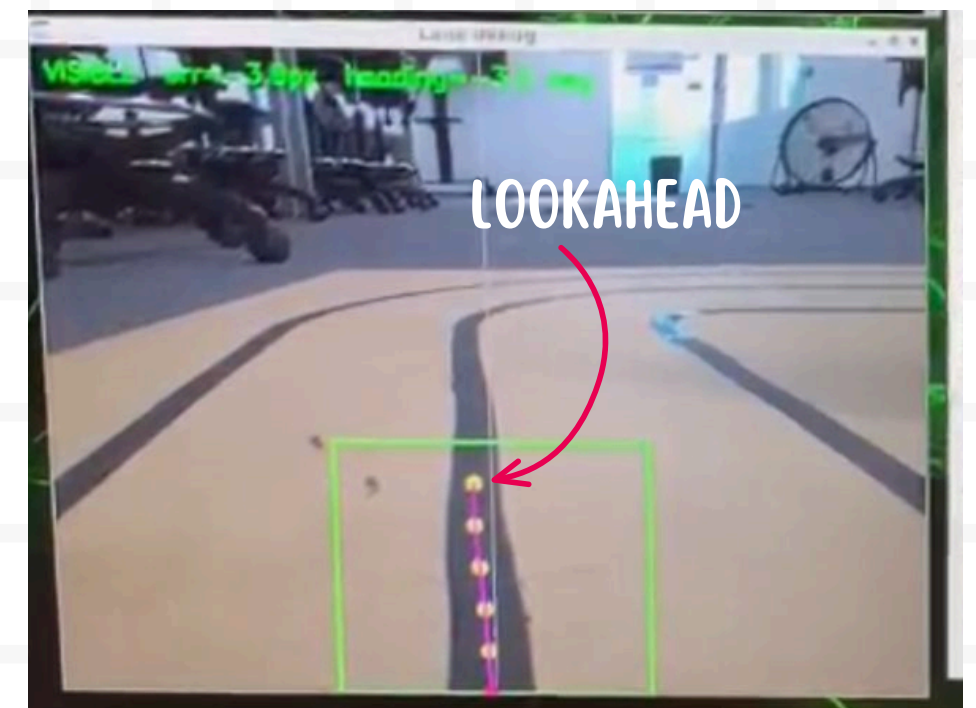
- la_y: coordenada y del lookahead.

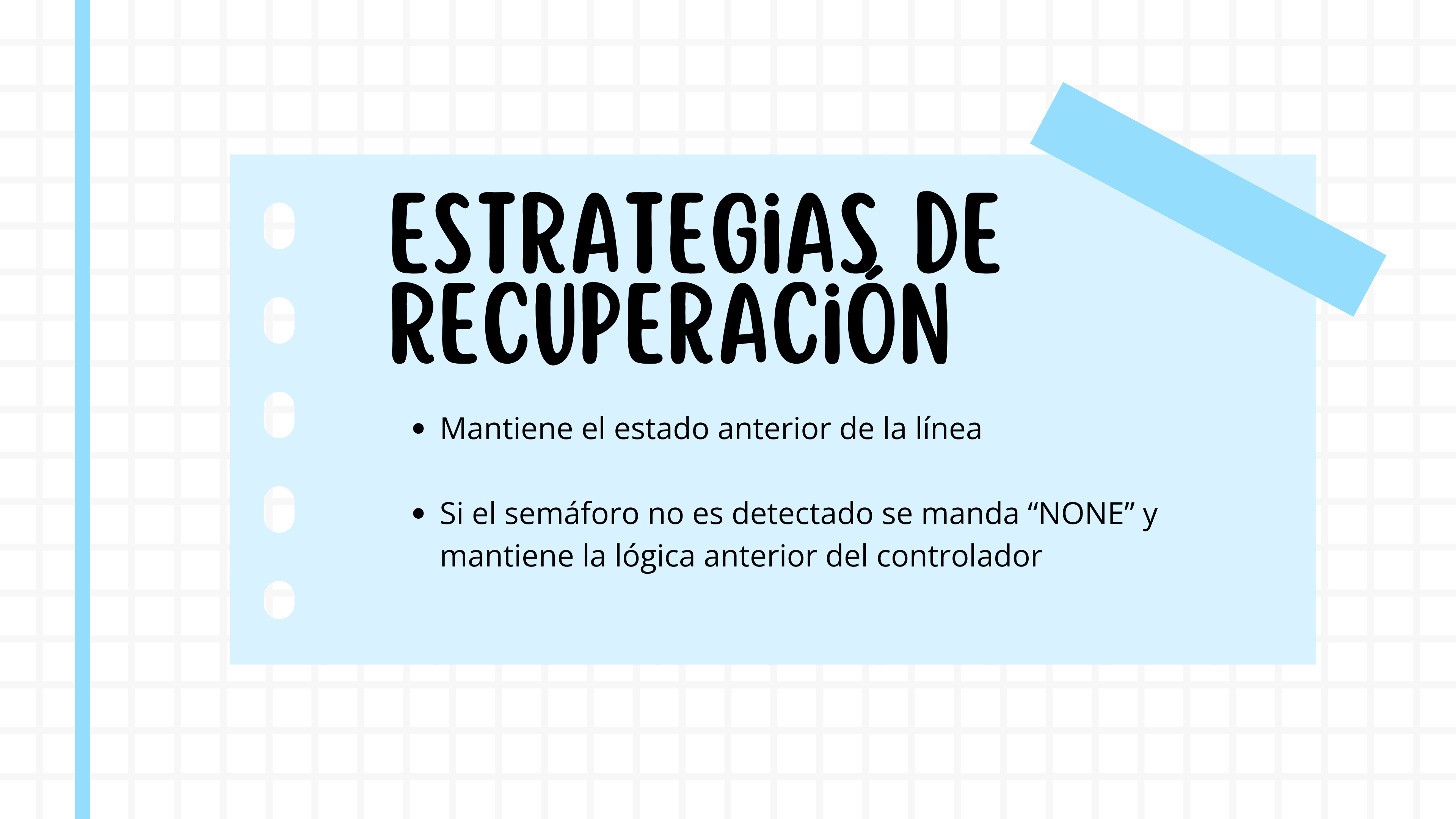
3

Con estos valores se obtiene el ángulo hacia el objetivo:

$$ALPHA = ATAN2(DX, DY)$$

Este ángulo indica hacia dónde debe girar el robot para volver a centrar la línea.



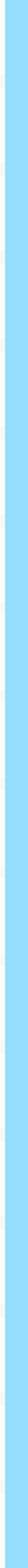


ESTRATEGIAS DE RECUPERACIÓN

- Mantiene el estado anterior de la línea
- Si el semáforo no es detectado se manda "NONE" y mantiene la lógica anterior del controlador

Conclusiones

Se desarrolló un pipeline de visión con control de manera correcta. El nodo de visión permite detectar tanto la línea como el color del semáforo en un mismo frame, mientras que el nodo controlador utiliza una estrategia tipo Pure Pursuit con una máquina de estados para generar los comandos de movimiento dependiendo de los colores del semáforo.



MUCHAS
GRACIAS

